

درس سیستم عامل

تمرین عملی سوم سوال اول

402105727

متین باقری

402105802

فاطمیما تیمارچی

1 و 2)

چون بخش 2 دربردارنده بخش یک بود این دو را با هم انجام دادم .

در کل خروجی این قسمت به صورت زیر میشه :

```
Timarchi-Bagheri@debian:~/Downloads/xv6-riscv-riscv$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ top
  PID   PPID  SIZE  STATE  NAME      init  STATUS
  1      0    16384  SLEEPING  init      USED
  2      1    20480  SLEEPING  sh        USED
  3      2    20480  RUNNING  top       USED
$ top -t
Process Tree:
├─ pid=1 (init) [SLEEPING]
│   └─ pid=2 (sh) [SLEEPING]
│       └─ pid=4 (top) [RUNNING]
$ top -tree
Process Tree:
├─ pid=1 (init) [SLEEPING]
│   └─ pid=2 (sh) [SLEEPING]
│       └─ pid=5 (top) [RUNNING]
```

برای درست کردن این درخت تابع top را به صورت زیر بازنویسی کردم :

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

char *states[] = {
    "UNUSED", "USED", "SLEEPING", "RUNNABLE", "RUNNING", "ZOMBIE"
};

void print_tree(struct process_data *procs, int n, int parent,
int depth) {
    for(int i = 0; i < n; i++) {
        if(procs[i].parent_id == parent) {
            for(int j = 0; j < depth; j++) printf("  ");
```

```

        printf("└─ pid=%d (%s) [%s]\n", procs[i].pid,
procs[i].name, states[procs[i].state]);

        print_tree(procs, n, procs[i].pid, depth + 1);
    }
}

int main(int argc, char *argv[])
{
    struct process_data all[64];
    int count = 0;
    int pid = 0;
    struct process_data p;

    while(1) {
        int r = next_process(pid, &p);
        if(r == 0)
            break;
        all[count++] = p;
        pid = p.pid;
    }

    if(argc > 1 && (!strcmp(argv[1], "-tree") || !strcmp(argv[1],
"-t"))) {
        printf("Process Tree:\n");
        print_tree(all, count, 0, 0);
    } else {
        printf("PID\tPPID\tSIZE\tSTATE\tNAME\t\tSTATUS\n");
        for(int i = 0; i < count; i++) {

            char *status = (all[i].state != 1) ? "USED" : "UNUSED";

```

```

        printf("%d\t%d\t%d\t%s\t%s\t%s\n", all[i].pid,
all[i].parent_id,
                all[i].head_size, states[all[i].state],
all[i].name, status);
    }
}

exit(0);
}

```

همان طور که در کد میبینید یک syscall به نام next_process اضافه کردم . برای اضافه کردن این syscall دقیقاً مطابق تمرین قبلی باید فایل های syscall.c و sysproc.c و usys.pl و sysproc.h و makefile و ... را ادیت میکردم تا این تابع را بتواند فراخوانی کند . من تمام فایل هایی که تغییر دادم رو در همین پوشه قرار دادم . اول مطابق صورت سوال باید struct داده شده در proc.h قرار میدادم که قرار دادم . حالا منطق اصلی این syscall یعنی next_process رو باید در kernel مینوشتیم . منطق آن هم به این صورت هست که میاد بر اساس pid ای که بهش میدیم رو همه فرایندها حلقه میرنه و فقط فرایندهای فعال رو چک میکنه و اگه فرایندی پیدا نکرد که صفر بر میگردد و اگر هم پیدا کرد struct ای که در بالا بهش اشاره کردم رو پر میکنه و اطلاعات اون process رو در حافظه ذخیره میکنه و سپس از kernel به user ارسال میکنه . کد این تکه در فایل sysproc.c به صورت زیر است(فایل کامل sysproc.c را ضمیمه کردم) :

```

uint64
sys_next_process(void)
{
    int before_pid;
    uint64 user_addr;
    struct process_data data;
    struct proc *p;
    struct proc *best = 0;

    argint(0, &before_pid);
    argaddr(1, &user_addr);

    for (p = proc; p < &proc[NPROC]; p++) {

```

```

    if (p->state != UNUSED && p->pid > before_pid) {
        if (best == 0 || p->pid < best->pid)
            best = p;
    }
}

if (best == 0)
    return 0;

data.pid = best->pid;
data.parent_id = best->parent ? best->parent->pid : 0;
data.head_size = best->sz;
data.state = best->state;
safestrcpy(data.name, best->name, sizeof(data.name));

if (copyout(myproc()->pagetable, user_addr, (char *)&data,
sizeof(data)) < 0)
    return -1;

return 1;
}

uint64
sys_getppid(void)
{
    return myproc()->parent ? myproc()->parent->pid : 0;
}

uint64
sys_sleep(void)
{
    int n;
    argint(0, &n);

    acquire(&tickslock);

```

```

uint ticks0 = ticks;
while (ticks - ticks0 < n) {
    if (killed(myproc())) {
        release(&tickslock);
        return -1;
    }
    sleep(&ticks, &tickslock);
}
release(&tickslock);
return 0;
}

```

حالا که همه توابع مورد نیاز رو اضافه کردیم و تونستیم اطلاعات هر فرایند رو به دست بیاریم طبق صورت سوال باید به صورت درختی هم نشون بدیم بنابراین کد top.c به صورت زیر میشه :

```

#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

char *states[] = {
    "UNUSED", "USED", "SLEEPING", "RUNNABLE", "RUNNING", "ZOMBIE"
};

void print_tree(struct process_data *procs, int n, int parent,
int depth) {
    for(int i = 0; i < n; i++) {
        if(procs[i].parent_id == parent) {
            for(int j = 0; j < depth; j++) printf("  ");

            printf("├─ pid=%d (%s) [%s]\n", procs[i].pid,
procs[i].name, states[procs[i].state]);

            print_tree(procs, n, procs[i].pid, depth + 1);
        }
    }
}

```

```

}

int main(int argc, char *argv[])
{
    struct process_data all[64];
    int count = 0;
    int pid = 0;
    struct process_data p;

    while(1) {
        int r = next_process(pid, &p);
        if(r == 0)
            break;
        all[count++] = p;
        pid = p.pid;
    }

    if(argc > 1 && (!strcmp(argv[1], "-tree") || !strcmp(argv[1],
"-t"))) {
        printf("Process Tree:\n");
        print_tree(all, count, 0, 0);
    } else {
        printf("PID\tPPID\tSIZE\tSTATE\tNAME\t\tSTATUS\n");
        for(int i = 0; i < count; i++) {

            char *status = (all[i].state != 1) ? "USED" : "UNUSED";
            printf("%d\t%d\t%ld\t%s\t%s\t%s\n", all[i].pid,
all[i].parent_id,
                all[i].head_size, states[all[i].state],
all[i].name, status);
        }
    }

    exit(0);
}

```

در این کد همون طور که مشاهده میکنید user اطلاعات همه فرایندهای فعال رو از طریق syscall ای به نام next_process میگیره و اگر در ترمینال top بزنیم بسته به اینکه مقادیر داخل struct چی هستند تمام ویژگیهای فرایندها رو از جمله running یا sleeping یا used یا ... در یک جدول چاپ میکنه . اگر هم کاربر دستور - top یا tree یا top -t رو بزنه این اطلاعات فرایندها رو به صورت درختی نشون میده . در نهایت هم خروجی به صورت زیر میشه :

```
Timarchi-Bagheri@debian:~/Downloads/xv6-riscv-riscv$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ top
  PID  PPID  SIZE  STATE  NAME      STATUS
  1      0   16384  SLEEPING  init      USED
  2      1   20480  SLEEPING  sh        USED
  3      2   20480  RUNNING  top       USED
$ top -t
Process Tree:
├─ pid=1 (init) [SLEEPING]
├─ pid=2 (sh) [SLEEPING]
├─ pid=4 (top) [RUNNING]
$ top -tree
Process Tree:
├─ pid=1 (init) [SLEEPING]
├─ pid=2 (sh) [SLEEPING]
├─ pid=5 (top) [RUNNING]
```

(3

در این بخش باید یک تست مینوشتیم که حالات مختلف رو تست کنه . در این قسمت با زدن فایل testTree اجرا میشه و خروجی به صورت زیر میشه :

```

$ testTree
Parent PID: 6
ZomUSSbIRUEeNLEePI cNND hIG INIGcd P IchID: 7lh, Pd child PaID: lIP9ID: red, 1 nP0,t: 6 PaPaI(rewrDnt: 8el:,ll e PxarenIntt: 6: 6t
an d b(simulatecd6 UoSE
me zoD state)
mble)

PID  PPID  SIZE  STATE  NAME  STATUS
1    0      16384 SLEEPING  init  USED
2    1      20480 SLEEPING  sh    USED
6    2      20480 RUNNING  testTree  USED
7    6      20480 ZOMBIE   testTree  USED
8    6      20480 RUNNABLE testTree  USED
9    6      20480 RUNNABLE testTree  USED
10   6      20480 SLEEPING testTree  USED

Process Tree:
├─ pid=1 (init) [SLEEPING] {USED}
├─ pid=2 (sh) [SLEEPING] {USED}
├─ pid=6 (testTree) [RUNNING] {USED}
│   ├── pid=7 (testTree) [ZOMBIE] {USED}
│   ├── pid=8 (testTree) [RUNNABLE] {USED}
│   ├── pid=9 (testTree) [RUNNABLE] {USED}
│   └─ pid=10 (testTree) [SLEEPING] {USED}
$

```

حالا منطق کد testTree به این صورت هست که اول در سمت user میاد یک سری فرایند های متفاوت ایجاد میکنه از جمله sleeping و running و ... و بعد اینکه همه رو ساخت میاد با next_processs وضعیت همه فرایند هارو از kernel میگیره . در وسط اجرای این کد ها دستور top و top -t زده میشه تا در حین اجرا بتونیم وضعیت فرایند های در حال اجرا رو ببینیم . حالا توضیح اینکه این حالت های مختلف چطور ایجاد میشن به صورت زیر است :

1- مثلا برای sleeping میاد دستور sleep رو میزنه (برای فرزند چهارم) که به حالت sleep بره و منتظر time ticket بمونه , برای اجرای دستور sleep چون این syscall رو نداشتیم مجبور شدیم این رو اضافه کنیم و طریقه اضافه کردنش هم دقیقا مثل syscall هایی در بخش قبلی یا تمرین قبلی بودند اضافه کردم و نکته جالبی نداشت باز هم فایل های تغییر یافته رو ر این پوشه ضمیمه کردم .

2- برای zombie هم میاد فرزند اول رو exit میکنه در حالی پدر این فرزند هنوز در wait هست .

3- برای runnable هم میایم فرزند دوم رو با استفاده از sleep منتظر نگه میداریم یعنی هنوز زنده هست و در حال اجرا نیست .

4- برای running هم میاد با استفاده از busy_work() میاد اون فرایند(فرزند سوم) رو همواره در حال اجرا نگه میداره .

برای used و unused هم دقیقا از همون struct استفاده میکنیم و انگار اینطوری میشه که تا وقتی فرزند زنده هست و kill نشده used میشه حتی zombie . پس نداریم چون unused برای خانه های جدول فرایند که خالی هستند برای همین همشون used میشوند .

حالا کد این قسمت رو هم ضمیمه کردم هم در ادامه قرار دادم . در این کد همان طور که میبینید من یک getppid تعریف کردم برای اینکه pid والد رو به دست بیارم . با زدن دستور ppid اگر پدر داشت که pid پدر رو بهمون میده

و اگر نداشت هم صفر برمیگردونه . دستور ppid هم منطق اصلیش رو به صورت زیر در sysproc.c اضافه کردم :

```
uint64
sys_getppid(void)
{
    struct proc *p = myproc();
    if(p->parent)
        return p->parent->pid;
    return 0;
}
```

کد فایل testTree به صورت زیر هست (این فایل در همین پوشه ضمیمه شده است) :

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

void busy_work() {
    volatile long i;
    for(i = 0; i < 500000000; i++);
}

void print_tree(struct process_data *procs, int n, int parent, int depth) {
    char *states[] = { "UNUSED", "USED", "SLEEPING", "RUNNABLE", "RUNNING", "ZOMBIE" };
    for(int i = 0; i < n; i++) {
        if(procs[i].parent_id == parent) {
            for(int j = 0; j < depth; j++) printf(" ");
            char *status;
            if(procs[i].state != 1)        status = "USED";
            else                          status = "UNUSED";

            printf("├─ pid=%d (%s) [%s] {%s}\n", procs[i].pid, procs[i].name,
                states[procs[i].state], status);
            print_tree(procs, n, procs[i].pid, depth + 1);
        }
    }
}
```

```

    }
}

int main(int argc, char *argv[]) {
    int i;

    printf("Parent PID: %d\n", getpid());

    //Test case 1: Zombie
    int zombie_pid = fork();
    if(zombie_pid == 0) {
        printf("Zombie child PID: %d, Parent: %d (will exit and become zombie)\n", getpid(), getppid());
        exit(0);
    }

    //Test case 2: USED
    int used_pid = fork();
    if(used_pid == 0) {
        printf("USED child PID: %d, Parent: %d (simulated USED state)\n", getpid(), getppid());
        sleep(3);
        exit(0);
    }

    // Test case 3: Running child ---
    int running_pid = fork();
    if(running_pid == 0) {
        printf("RUNNING child PID: %d, Parent: %d\n", getpid(), getppid());
        busy_work();
        exit(0);
    }

    // Test case 4: Sleeping
    int sleeping_pid = fork();
    if(sleeping_pid == 0) {
        printf("SLEEPING child PID: %d, Parent: %d\n", getpid(), getppid());

```

```

    sleep(5);

    exit(0);
}

sleep(2);

struct process_data all[64];

int count = 0;

int pid = 0;

struct process_data p;

while(1) {

    int r = next_process(pid, &p);

    if(r == 0)

        break;

    all[count++] = p;

    pid = p.pid;
}

printf("\nPID\tPPID\tSIZE\tSTATE\tNAME\tSTATUS\n");

char *states[] = { "UNUSED", "USED", "SLEEPING", "RUNNABLE", "RUNNING", "ZOMBIE" };

for(i = 0; i < count; i++) {

    char *status;

    if(all[i].state != 1)        status = "USED";

    else                        status = "UNUSED";

    printf("%d\t%d\t%d\t%s\t%s\t%s\n", all[i].pid, all[i].parent_id,

        all[i].head_size, states[all[i].state], all[i].name, status);
}

printf("\nProcess Tree:\n");

print_tree(all, count, 0, 0);

for(i = 0; i < 4; i++)

    wait(0);

exit(0);

```

}